

AI for Geopolitics — État de l’art & positionnement du projet

Laury Kibamba

Version 1.0 — 30 juin 2026

Table of Contents

Une simulation géopolitique agentique où des pays-agents LLM, contraints par des données réelles, réagissent par rounds sous la supervision d’un Game Master — et un terrain d’application de bout en bout de la feuille de route de l’AI Engineer.

1. Résumé exécutif

AI for Geopolitics est un projet de plateforme de simulation et d’analyse géopolitique dans laquelle chaque pays est modélisé comme un agent d’intelligence artificielle. Ces agents ne sont pas de simples étiquettes ni de simples invites « joue le rôle de la Chine » : ils sont contraints par des paramètres structurés inspirés du monde réel — économie, armée, ressources, alliances, stabilité politique, technologie, commerce, idéologie — et dotés d’une mémoire. Un **Game Master** (humain ou IA) déclenche des événements ; les pays-agents analysent, décident, négocient par messages visibles, forment ou rompent des alliances, et font évoluer un état du monde par rounds. Une couche **RAG** ancre les décisions dans des sources documentaires, et un **moteur de risque** structure des signaux (escalade, perturbation économique, fracture d’alliance) de manière explicable.

Le projet poursuit un **double objectif**. Sur le fond, produire un simulateur géopolitique crédible, sourcé et jouable. Sur la forme, servir de **vecteur d’apprentissage** : c’est un projet capstone qui exerce, de bout en bout, les compétences d’un *AI Engineer* — fonctionnement des modèles, embeddings et RAG, systèmes agentiques et MCP, ingénierie de prompt, choix et déploiement de modèles, sécurité et observabilité (cf. section 10.5). Une contrainte structurante en découle : faire tourner l’essentiel **en local** sur un poste personnel (NVIDIA RTX 2060 Super 8 Go, Ryzen 7 3700X, 32 Go de RAM), en identifiant les goulots d’étranglement matériels (section 8). Cet axe d’apprentissage est exigeant et concret : construire un **service d’inférence LLM optimisé, écrit en Python**, pour saisir la réalité physique de l’IA générative et maximiser le rendement d’un matériel coûteux ; démontrer la **maîtrise de la conteneurisation** (Docker, Kubernetes) ; et tenir une **discipline d’ingénierie** — code propre et orienté objet, tests, usage rigoureux de Git, solutions simples plutôt qu’usines à gaz (cf. sections 8.5 et 10.6).

Ce document fait l’état de l’art du projet et son positionnement. Il (1) formule les **problématiques** à résoudre, (2) cartographie ce qui existe déjà — recherche, frameworks, jeux de données, produits — autour de chacune, et (3) **positionne** AI for Geopolitics dans ce paysage. Trois constats orientent la suite. D’abord, la **simulation géopolitique par**

agents LLM est un champ de recherche actif : de WarAgent (2023) à des travaux de 2025–2026 sur l’alignement comportemental et la calibration du risque, la communauté a posé des fondations mais aussi documenté des écueils (biais d’escalade, biais géopolitiques, difficulté de validation). Ensuite, **l’écosystème agentique a mûri** (LangGraph, Model Context Protocol, données ouvertes), rendant l’architecture réaliste. Enfin, **le différenciateur du projet réside dans la combinaison** — agents contraints par des données + RAG sourcé + Game Master + négociation visible + risque explicable + déploiement reproductible — et dans la rigueur de validation qui l’accompagne.

2. Cadrage du projet et problématiques

2.1 Le projet en bref

La phrase la plus fidèle : **un serious game / simulateur géopolitique agentique, nourri par des données réelles, où des pays-agents LLM mais contraints par des paramètres structurés réagissent à des crises, négocient, forment des alliances, mettent à jour un état du monde, et produisent des analyses sourcées via RAG.** La boucle de jeu est événementielle : état initial → événement annoncé par le Game Master → analyse de chaque pays → décisions → négociations et alliances → conséquences économiques, militaires et diplomatiques → mise à jour de l’état du monde → résumé du round → round suivant.

2.2 Une nature pluridisciplinaire

Le projet se situe à l’intersection de quatre familles techniques, ce qui conditionne l’état de l’art à couvrir :

- **Simulation multi-agent** : agents-pays, rounds, interactions, monde qui évolue (sections 4 et 5.2).
- **RAG géopolitique** : sources, récupération, réponses sourcées, mémoire documentaire (section 7).
- **MLOps / AI engineering** : jeux de données, pipeline robuste, backend, évaluation, déploiement, exécution locale (sections 5, 6 et 8).
- **Jeu stratégique / serious game** : Game Master, alliances, négociation, interface visible (sections 4 et 9).

2.3 Les problématiques structurantes

De la vision se dégagent neuf problématiques. Elles servent de fil conducteur : chaque section y répond, et la section 11 les relie aux briques à construire.

- **P1 — Crédibilité comportementale sans simple prompt engineering.** Comment faire émerger des décisions d’État plausibles à partir d’un mélange profil + données + mémoire + contraintes + raisonnement, plutôt que d’un seul system prompt ?
- **P2 — Ancrage dans des données réelles.** Comment construire et contraindre les pays à partir de jeux de données (économie, militaire, énergie, commerce) plutôt que de les « inventer » au prompt ?

- **P3 — Modélisation des conséquences et du risque.** Comment faire en sorte que les décisions aient des effets, et structurer des scores de risque défendables — sans prétendre « prédire la guerre » ?
- **P4 — Traçabilité et ancrage documentaire.** Comment garantir que les analyses sont sourcées, citables et évaluables (qualité du retrieval) ?
- **P5 — Mémoire multi-niveaux.** Comment stocker et exploiter une mémoire du monde, par pays, conversationnelle (diplomatie) et documentaire ?
- **P6 — Orchestration et passage à l'échelle.** Comment orchestrer les agents (machine à états, MCP, broker) et déployer (Docker, Kubernetes) sans sur-ingénierie prématurée ?
- **P7 — Évaluation et validation.** Comment évaluer une simulation dont la « vérité terrain » est partielle, et démontrer qu'elle apprend quelque chose plutôt qu'elle n'hallucine ?
- **P8 — Éthique, biais et usage dual.** Comment gérer les biais géopolitiques des modèles, le biais d'escalade, et le caractère sensible d'un simulateur de conflits ?
- **P9 — Exécution locale et contraintes matérielles.** Comment faire tourner l'essentiel du système sur un poste personnel (GPU 8 Go, 32 Go de RAM), identifier les goulots d'étranglement (VRAM, cache KV) et arbitrer entre inference locale et API ? Et comment en faire un **service d'inférence Python** optimisé, conteneurisé et mesurable ?

3. Méthodologie de l'état de l'art

La revue couvre cinq corpus : (a) la littérature académique sur la simulation de relations internationales et le wargaming par LLM ; (b) les frameworks d'orchestration agentique et de simulation multi-agent ; (c) les jeux de données géopolitiques et les mesures de risque ; (d) l'état de l'art du RAG et de la mémoire d'agents ; (e) les outils et arbitrages d'exécution locale de LLM. Les sources combinent articles de recherche (arXiv, FAccT, UIST, AER), documentation officielle (Anthropic, Microsoft Research, GDELT, SIPRI), comparatifs d'ingénierie récents, et la *Feuille de route de l'AI Engineer* (Tomes 2 à 5). La recherche a été conduite en juin 2026 et privilégie les travaux 2023–2026, le champ évoluant vite. Chaque affirmation factuelle renvoie à une référence numérotée en fin de document.

4. Travaux connexes : simulation géopolitique par agents LLM

Cette section répond surtout à P1 (crédibilité), P3 (risque) et P8 (biais/escalade). Elle montre que le cœur du projet — des pays-agents qui décident et négocient — a déjà des précédents sérieux, à la fois rassurants (faisabilité) et exigeants (il faut se différencier).

4.1 Négociation stratégique : CICERO (Meta, Science 2022)

CICERO est le premier agent à atteindre un niveau humain au jeu **Diplomacy**, qui mêle coopération et compétition et où la négociation en langage naturel entre sept puissances est centrale [3]. Il combine un modèle de langage avec de la planification et de

l'apprentissage par renforcement, en inférant croyances et intentions des autres joueurs pour générer un dialogue au service de ses plans — plus du double du score moyen humain sur 40 parties, top 10 %. **Enseignement** : la négociation textuelle visible (feature explicitement voulue) est un problème dur mais traité ; un dialogue crédible suppose un raisonnement stratégique sous-jacent, pas seulement de la génération de texte. Cela conforte le choix d'une architecture **hybride** (LLM + état + contraintes) plutôt que purement conversationnelle.

4.2 Simulation historique de conflits : WarAgent (2023)

WarAgent est le travail le plus proche du projet : un système multi-agent LLM qui simule les pays participant à des conflits historiques (deux guerres mondiales, Royaumes combattants) pour étudier les déclencheurs de guerre [1]. Son architecture repose sur des **country agents**, des **secretary agents** (contrôle de cohérence et de validité des actions) et un cadre « Board and Stick » modélisant relations internationales et dynamique interne des États.

Enseignements : (1) la séparation country agent / secretary agent valide l'idée d'un agent de contrôle distinct du raisonnement — proche du Game Master/arbitre du projet ; (2) WarAgent reste centré sur le texte et l'histoire, **sans couche de données quantitatives structurées** (PIB, dépenses militaires, dépendances commerciales) ni RAG sourcé : c'est précisément l'espace que P2 et P4 ouvrent pour se différencier.

4.3 Agents génératifs et simulation sociale

Les **Generative Agents** de Stanford (Park et al., UIST 2023) ont popularisé une architecture de mémoire désormais standard : un flux d'observations en langage naturel, une **synthèse périodique en réflexions**, et une **récupération dynamique** des souvenirs pertinents pour planifier l'action [4]. C'est directement réutilisable pour la mémoire par pays (P5) : promesses, trahisons, relations bilatérales, objectifs. Plus largement, le courant **GABM (Generative Agent-Based Modeling)** applique les LLM à la simulation sociale, mais des revues critiques de 2025 pointent un risque majeur : la quasi-totalité des études reposent sur du prompting zero-shot sans calibration, réduisant la calibration à de l'ingénierie de prompt, avec un danger de biais [7][24]. **La validation est identifiée comme le défi central de la simulation sociale générative** [24] — ce qui justifie de traiter P7 comme une exigence de premier ordre.

4.4 Sécurité et escalade : un signal d'alerte documenté

Le travail de **Rivera et al. (FAccT 2024)** a fait date : dans des wargames simulés, les cinq LLM testés présentent tous des formes d'escalade difficiles à prédire, parfois justifiées par des logiques de dissuasion ou de première frappe ; les auteurs recommandent une grande prudence avant de déléguer des décisions stratégiques à des agents autonomes [2]. Un travail de 2025, « **Red Lines and Grey Zones in the Fog of War** », propose un **benchmark** du risque légal, du préjudice moral et du biais régional dans la décision militaire par LLM [6]. **Conséquence pour P3 et P8** : le projet doit se positionner comme un outil d'analyse

de signaux de risque **explicables**, doté de garde-fous, et non comme un oracle prédictif ni un assistant à la décision létale.

4.5 La vague 2025–2026 : alignement, calibration, outils

« **LLMs as Strategic Actors** » (mars 2026) évalue six LLM de pointe face à des résultats humains sur quatre scénarios de crise réels, en mesurant l'alignement des actions, la **calibration du risque** (sévérité des actions) et le **cadrage argumentatif** ancré dans la théorie des relations internationales [5]. Résultats : les modèles approchent les schémas de décision humains aux premiers rounds mais **divergent dans le temps**, adoptant des profils distincts ; leurs justifications affichent un **cadrage normatif-coopératif** marqué (stabilité, coordination, atténuation du risque) avec un raisonnement adverse limité. Pour le projet, c'est une validation méthodologique (mesurer alignement, sévérité, cadrage est faisable) doublée d'un avertissement : la dérive temporelle et le biais coopératif doivent être suivis explicitement.

Côté outils, des projets ouverts apparaissent, tel **Geopol-Modeller** (approche « Snowglobe »), simulation multi-acteurs LLM pour la prospective, montrant que chaque étape d'un wargame textuel peut être conduite par des LLM [9]. Ils confirment l'appétence pour ce type de système, tout en restant mono-bloc, peu outillés en données structurées et faiblement évalués.

4.6 Synthèse : enseignements pour le projet

- **Faisabilité prouvée** : pays-agents (WarAgent), négociation crédible (CICERO), mémoire d'agent (Generative Agents) sont déjà démontrés.
- **Espace de différenciation** : peu de systèmes couplent agents + **données quantitatives structurées** + **RAG sourcé** + **évaluation rigoureuse**. C'est l'angle du projet (P2, P4, P7).
- **Exigences non négociables** : garde-fous anti-escalade et explicabilité du risque (P3, P8) ; suivi de la dérive comportementale et des biais.

5. Briques techniques : orchestration agentique

Cette section répond à P1 et P6. Le projet a besoin d'orchestrer un cycle par rounds avec état persistant et arbitrage, d'exposer des outils aux agents, et éventuellement de découpler les échanges via un broker.

5.1 LangGraph vs AutoGen vs CrewAI

Les trois frameworks dominants de 2025–2026 incarnent trois philosophies : CrewAI met l'accent sur les rôles, LangGraph sur la structure du workflow (graphe d'états), AutoGen sur la conversation [21]. Le Tome 3 de ta feuille de route privilégie d'ailleurs **LangGraph** pour l'orchestration multi-agents [29].

Framework	Paradigme	Forces	Limites	Pertinence projet
-----------	-----------	--------	---------	-------------------

LangGraph	Graphe d'états	État explicite, persistance (PostgresSaver), human-in-the-loop, branchements	Courbe d'apprentissage, plus de code	Élevée : le moteur de rounds est un graphe d'états BASE→RECALL→FALLBACK
AutoGen	Conversation	Dialogues multi-agents flexibles, recherche, v1.0 GA	Coûteux en temps réel, contrôle d'état faible	Moyenne : prototypage de la diplomatie
CrewAI	Rôles + Flows	Démarrage rapide (YAML), rôles clairs, monitoring	Moins flexible pour la logique complexe	Moyenne : agents-pays « à rôle » en V1

Recommandation : adopter **LangGraph** comme moteur d'orchestration (l'état du monde et la machine à états s'y modélisent directement, persistance Postgres native), quitte à prototyper la couche diplomatique avec une logique plus légère. Choix non exclusif : les agents restent des fonctions appelant des outils.

5.2 Mesa / Mesa-Geo : la simulation multi-agent « classique »

Mesa est le framework de référence pour l'agent-based modeling en Python, avec son extension géospatiale **Mesa-Geo** [22]. Il n'orchestre pas des LLM mais fournit l'ossature éprouvée d'une simulation à pas de temps (scheduler, collecte de données, visualisation). Pertinent en **Phase 0** (moteur déterministe sans LLM) pour valider la boucle de rounds, le WorldState et le moteur de conséquences avant d'introduire le raisonnement LLM — l'ordre sain recommandé.

5.3 Le Model Context Protocol (MCP)

Le projet exprimait une préférence pour « MCP au centre plutôt que REST ». Cette intuition est désormais alignée avec le marché : annoncé par Anthropic en novembre 2024, **MCP est devenu en un an le standard de fait** pour connecter les agents aux outils et aux données, adopté par OpenAI, Google DeepMind, Microsoft et AWS, avec des milliers de serveurs, puis placé sous gouvernance neutre fin 2025 [20]. Le Tome 3 détaille précisément son architecture (hôte, client, serveur, primitives outils/ressources/prompts, couche JSON-RPC) [29]. Concrètement, les capacités des agents-pays (get_country_state, get_alliances, propose_action, send_diplomatic_message, resolve_round, search_documents) peuvent être exposées comme **outils MCP**, l'API REST restant dédiée à l'interface.

Avertissement : la qualité des descriptions d'outils MCP impacte fortement l'efficacité des agents — un facteur de réussite documenté.

5.4 Broker Redis / Kafka

Pour découpler la distribution des événements et la collecte des décisions : **Redis** (Streams / Pub-Sub) suffit largement en V1 — simple, déjà utile comme cache et mémoire courte. **Kafka** n'apporte un bénéfice (rejouabilité, partitionnement, débit) qu'à un stade ultérieur, multi-services. Recommandation : **Redis d'abord**, Kafka seulement si le besoin d'évent sourcing se confirme.

6. Briques techniques : données géopolitiques

Cette section répond à P2. L'enjeu est de remplir le CountryState et de générer des événements à partir de sources ouvertes reconnues, plutôt que d'inventer les pays au prompt.

6.1 Indicateurs structurés (profil des pays)

- **World Bank Open Data** : PIB, croissance, population, indicateurs macro — bloc « économie » [16].
- **SIPRI** : dépenses militaires, transferts d'armes, embargos — bloc « armée » [14].
- **OECD (Observatory of Economic Complexity)** : flux commerciaux, dépendances export/import — bloc « commerce » [17].
- **Votes à l'ONU** (jeux Voeten / Bailey-Strezhnev) : alignements diplomatiques, proximité idéologique — pour initialiser alliances et idéologie.

6.2 Événements et conflits

- **GDELT** : base temps réel (toutes les 15 min) de l'actualité mondiale en 65+ langues, avec son **Global Knowledge Graph** (acteurs, lieux, thèmes, tonalité) — idéal pour alimenter le Game Master en événements réels et le RAG en contexte [11].
- **ACLED** : événements de violence politique géolocalisés (dates, types, acteurs, victimes) depuis 1997 [12].
- **UCDP (Uppsala)** : le plus ancien programme de données sur la violence organisée, séries longues et comparables [13].
- **Correlates of War** : guerres inter-étatiques et différends militarisés depuis le XIX^e siècle — amorçage historique [15].

6.3 Mesures de risque existantes : l'indice GPR

L'**indice de risque géopolitique (GPR)** de Caldara et Iacoviello (AER, 2022) mesure le risque par comptage d'articles de presse relatifs aux tensions, organisé en huit catégories (menaces de guerre, renforcements militaires, menaces nucléaires, terrorisme...), disponible mensuellement depuis 1900 [10]. C'est une **référence méthodologique directe** pour le moteur de risque (P3) : il montre comment construire un signal textuel transparent et reproductible — la posture « structurer des signaux et expliquer les facteurs » plutôt que « prédire la guerre ».

6.4 Correspondance jeux de données → CountryState

Source	Type	Dimension alimentée	Accès
World Bank	Indicateurs	Économie (PIB, croissance, dette)	API ouverte, CSV
SIPRI	Indicateurs	Armée (budget défense, armes)	Bases téléchargeables
OECD	Réseau commerce	Commerce, dépendances import/export	API, datasets
Votes ONU	Vecteurs de vote	Alliances, idéologie, alignement	Datasets académiques
GDELT	Événements + GKG	Événements, tensions, contexte RAG	BigQuery, fichiers 15 min
ACLED / UCDP	Événements conflits	Conflits actifs, instabilité	API / téléchargement
Correlates of War	Conflits historiques	Amorçage historique, doctrines	Datasets CSV
GPR Index	Indice de risque	Référence du risk engine	Séries publiques

6.5 Gouvernance des données

Chaque source impose des contraintes (P8) : **licences** (ACLED et SIPRI ont des conditions spécifiques), **fraîcheur** (GDELT temps réel vs séries annuelles), **biais de couverture** (GDELT et GPR sur-représentent la presse anglophone), **comparabilité** (UCDP et ACLED ne codent pas la violence identiquement). Un document `data_governance.md` recensant origine, licence, fréquence et limites de chaque jeu de données est recommandé dès la Phase 4.

7. Briques techniques : RAG géopolitique et mémoire

Cette section répond à P4 et P5. Bonne nouvelle : le pipeline RAG déjà imaginé — query rewriting/expansion, semantic chunking, hybride Chroma + BM25, RRF, normalisation, reranking cross-encoder, self-querying, machine à états BASE→RECALL→FALLBACK — correspond très précisément à l'état de l'art 2025–2026 et au Tome 2 de ta feuille de route [28].

7.1 Recherche hybride, fusion et reranking

Le patron dominant : exécuter en parallèle une recherche **lexicale (BM25)** et **dense (vectorielle)**, fusionner les classements via **Reciprocal Rank Fusion (RRF)** — approche popularisée sous le nom de **RAG-Fusion** [18] — avant un **reranking par cross-encoder**. Ce reranking est l'amélioration au meilleur rapport coût/bénéfice : un pipeline typique récupère un top-50 hybride, le reranke en top-5, puis génère, avec des gains de 15–30 % sur les métriques RAGAS. Le pipeline du projet est donc conforme ; seul reste à acter le choix **Elasticsearch** vs simple BM25 — non indispensable en V1.

7.2 De la machine à états à l'Agentic RAG

La machine **BASE** → **RECALL** → **FALLBACK** (recherche normale ; filet large avec expansion + hybride + reranking si signal faible ; clarification sinon), limitée à deux passes, est une instance de ce que la littérature nomme **Agentic RAG** : une boucle de raisonnement qui décide dynamiquement d'un retrieval multi-étapes, d'un appel d'outils, ou d'une auto-correction avant de générer. Le projet anticipait une tendance de fond ; il suffit de la nommer et de la mesurer.

7.3 GraphRAG : exploiter les relations

La géopolitique est relationnelle (pays, acteurs, alliances, événements). **GraphRAG** (Microsoft Research, 2024) construit automatiquement un graphe de connaissances entités-relations à partir de texte, puis exploite cette structure et des « communautés » (algorithme de Leiden) pour répondre à des questions globales que la recherche vectorielle gère mal [19]. Option à fort potentiel pour relier le graphe d'alliances/tensions du WorldState au RAG documentaire (le Tome 2 le cite parmi les techniques avancées, avec RAPTOR) [28]. À considérer en Phase 3+.

7.4 Mémoire d'agents multi-niveaux

Quatre niveaux, chacun avec un stockage adapté (P5) : **mémoire du monde** (état structuré) et **relations pays-pays** en PostgreSQL ; **mémoire conversationnelle** (négociations) en buffer + résumés ; **mémoire par agent** (promesses, trahisons, objectifs) selon le schéma observation → réflexion → récupération des Generative Agents [4], en résumé + vector store ; **mémoire documentaire** (chunks, métadonnées, citations) en base vectorielle + index lexical. L'historique des rounds gagne à être un **event store** (table append-only Postgres) pour la rejouabilité et l'audit.

7.5 Évaluation du retrieval et de la génération

Les métriques prévues — **recall@k**, **precision@k**, **MRR** — sont les bonnes pour le retrieval. Côté génération sourcée, l'écosystème ajoute des cadres d'évaluation (RAGAS pour fidélité/pertinence, approche « nugget » de TREC RAG pour la couverture factuelle). Adopter un jeu de questions de référence avec sources attendues permet de répondre à P7 de façon mesurable.

8. Exécution locale : stratégies et goulots d'étranglement matériels

Cette section répond à P9. Contrainte : faire tourner l'essentiel du système sur ton poste — **NVIDIA RTX 2060 Super (8 Go GDDR6, architecture Turing, bus 256-bit $\approx$ 448 Go/s, compute capability 7.5), Ryzen 7 3700X (8 cœurs / 16 threads, Zen 2), 32 Go de RAM**. L'objectif est double : apprendre tout le stack en local, et savoir où sont les limites pour arbitrer local $\rightarrow$ API (l'arbitrage central du Tome 5 [31]).

8.1 Profil de charge des composants

Composant	Ressource dominante	Tient en local (8 Go VRAM / 32 Go RAM) ?
Inference LLM (agents)	VRAM (poids + cache KV)	Oui : 7–8B quantifié Q4 ; 13–14B en offload (lent)
Embeddings (RAG)	CPU / petit GPU	Oui, facile (bge/e5/nomic), batch hors-ligne
Reranker cross-encoder	CPU / petit GPU	Oui sur CPU (modèle petit)
Vector DB (Chroma)	RAM / disque	Oui (corpus modéré)
Services (Postgres, Redis)	RAM / CPU	Oui (légers)
Orchestration / app	CPU / RAM	Oui

Lecture clé : **un seul composant sature le GPU — l'inference des agents**. Tout le reste tient confortablement sur le CPU (16 threads) et les 32 Go de RAM.

8.2 Le goulot d'étranglement n°1 : la VRAM (poids + cache KV)

Sur 8 Go, un modèle **7–8B quantifié Q4_K_M** occupe $\approx$ 4,5–5,5 Go ; il reste $\approx$ 2,5–3 Go pour le **cache KV** et l'overhead. Or le cache KV grandit avec la **longueur de contexte** : des prompts longs (état du monde + mémoire + contexte RAG + espace d'action) multipliés par 6–8 agents le gonflent vite. **Le vrai plafond n'est souvent pas la taille du modèle mais la longueur de contexte**. Un 13–14B Q4 ($\approx$ 7,4 Go de poids) ne laisse plus de place au cache $\rightarrow$ offload CPU $\rightarrow$ chute à quelques tokens/s. Conclusion : viser 7–8B, contexte court, prompts élagués.

Rang	Goulot	Pourquoi	Parade
1	VRAM (poids + KV)	Détermine taille du modèle, longueur de contexte, co-résidence	Quantization Q4, contexte court (4–8k), prompts élagués, un seul modèle, flash-attention
2	Bande passante / compute GPU	Détermine les tokens/s ($\approx$ 448 Go/s, Turing = correct mais pas	Petits modèles, sorties courtes (JSON capé), prompt caching

		récent)	
3	RAM 32 Go	Offload CPU + services	Confortable ; plafonner la mémoire des conteneurs
4	CPU (16 threads)	Inference offloadée, embeddings, BM25	Atout : embeddings + reranking sur CPU
5	Disque / I/O	Fichiers GGUF (5–9 Go), index vectoriel	SSD, charger le modèle une seule fois

Ordre de grandeur attendu : un 7–8B Q4 sur RTX 2060 Super génère environ **25–45 tokens/s**. Un round de 6–8 agents, chacun produisant 300–500 tokens (raisonnement + JSON), prend de l'ordre de **quelques dizaines de secondes à 1–2 minutes**. C'est acceptable pour un jeu **par tours** (pas du temps réel) — et cohérent avec la nature « rounds » du projet.

8.3 Stratégies pour tenir en local

- **Local-first hybride (arbitrage Tome 5 [31])** : petit modèle local pour le gros des décisions ; **API frontière (Claude / GPT)** pour le Game Master, l'arbitrage des rounds difficiles, et le « juge » d'évaluation. Une couche d'abstraction (type LiteLLM ou interface provider) permet de basculer local → API par appel.
- **Quantization GGUF Q4_K_M / Q5** : compromis taille/qualité maîtrisé. Turing supporte bien INT4/INT8 (pas de FP8, réservé aux cartes récentes).
- **Budget de contexte** : résumer l'état du monde, ne garder que le **top-k 3–5** après rerank, mémoire en **résumé glissant** (Generative Agents), max_tokens capés, **sortie JSON structurée** (Tome 4 [30]) → réduit le cache KV (donc la VRAM) et la latence.
- **Agents séquentiels + cache de préfixe** : un seul modèle en VRAM → traiter les pays un par un ; réutiliser le cache KV du **préfixe commun** (état du monde) via le prompt cache de llama.cpp/Ollama pour ne pas le recalculer à chaque agent.
- **Décharger sur le CPU** ce qui est cheap (embeddings, reranking, BM25) ; **réserver le GPU au génératif**. Embedder le corpus une seule fois, hors-ligne.
- **Activer flash-attention** (llama.cpp --flash-attn) : réduit la mémoire du cache KV sur Turing.
- **Right-size la simulation (MVP)** : 6–8 acteurs, 1 zone, peu de rounds = volume de tokens maîtrisé. La discipline MVP est aussi une mitigation matérielle.
- **Mesurer avant d'optimiser (observabilité Tome 3 [29])** : nvidia-smi (VRAM), tokens/s, latence par round, tracing Langfuse/LangSmith. Trouver le vrai goulot empiriquement.
- **Windows** : **Ollama** natif comme baseline rapide (avant le service Python custom de la section 8.5) ; **WSL2** (GPU passthrough) pour l'outillage Linux (vLLM, etc.) ; **Docker Desktop** (backend WSL2) pour les services ; régler .wslconfig pour partager les 32 Go sans étouffer le modèle.

- **Conteneurisation avec limites de ressources** (Compose) ; **kind / Kubernetes** pour la *phase d'apprentissage infra*, pas pour héberger le modèle sur le laptop (le GPU ne se partage pas bien dans kind).

8.4 Pile locale recommandée (RTX 2060 Super 8 Go / Ryzen 7 3700X / 32 Go)

Brique	Choix recommandé	Où ça tourne
Inference agents	Service Python (FastAPI + backend optimisé : llama-cpp-python / ExLlamaV2 / vLLM) servant un 7–8B quantifié ; Ollama en baseline	GPU (8 Go)
Embeddings	bge-small / e5 / nomic-embed	CPU (16 threads)
Reranking	bge-reranker-base / MiniLM cross-encoder	CPU
Vector DB + lexical	Chroma + BM25 (rank_bm25) ; Elasticsearch optionnel	RAM / disque
Stores	PostgreSQL (état + event store), Redis (cache + broker)	Docker, mémoire plafonnée
Modèles « lourds »	API frontière (Claude / GPT) pour Game Master, rounds difficiles, juge d'éval	API
Observabilité	Langfuse / LangSmith + métriques GPU	local

Cette pile permet d'exercer **tout** le curriculum AI Engineer en local, avec le goulot (VRAM / cache KV) compris et maîtrisé, et un arbitrage local/API assumé.

8.5 Le service d'inférence LLM en Python (objectif d'étude)

Un **objectif d'apprentissage explicite** du projet est de construire un **service d'inférence LLM optimisé, écrit intégralement en Python** — non pour réinventer un moteur bas niveau, mais pour **comprendre la réalité physique de l'IA générative** (VRAM, cache KV, quantization, batching, offload) et apprendre à **maximiser le rendement d'un matériel coûteux**. Concrètement : un microservice **FastAPI** exposant une API compatible OpenAI, adossé à un backend optimisé, et instrumenté.

Le choix du backend illustre l'arbitrage simplicité ↔ performance (rester pédagogique, éviter l'usine à gaz) :

Backend Python	Intérêt pédagogique	Sur 8 Go Turing
Transformers +	Le plus explicite : tokenizer,	OK pour 7B 4-bit, lent mais

bitsandbytes (4-bit) + accelerate	boucle de génération, device_map (offload) visibles	clair
llama-cpp-python (GGUF)	Simple, offload CPU/GPU fin, prompt cache, serveur intégré	Idéal pour démarrer (rentre en 8 Go)
ExLlamaV2 (EXL2 / GPTQ)	Très rapide sur GPU grand public, cache KV efficace	Excellent rapport perf sur Turing
vLLM (paged attention, batching continu)	Production : batching, débit, API OpenAI	Possible mais 8 Go tendu (contexte court)

Ce que le service doit faire apprendre et mesurer : l'occupation VRAM (poids vs cache KV), l'effet de la longueur de contexte, le gain de la quantization (Q4/Q5/EXL2), l'intérêt du **batching** quand plusieurs agents tournent, l'**offload** GPU→CPU→RAM, et le débit (tokens/s) vs la latence. C'est exactement la « réalité physique » visée. Progression saine : **llama-cpp-python** d'abord (ça tient, c'est simple), puis **ExLlamaV2** / **vLLM** pour optimiser — chaque étape mesurée (section 8.2) avant d'être adoptée.

Discipline : ce service est un **objet orienté objet propre** (une interface InferenceBackend, des implémentations interchangeables), **testé**, **conteneurisé** (image GPU dédiée) et versionné — un cas d'école des principes de la section 10.6.

9. Paysage produit : « build vs buy »

Des produits commerciaux d'intelligence du risque géopolitique existent et délimitent ce que le projet n'a pas à refaire. **RANE / Stratfor** fournit analyse experte et prévisions adossées à 25 ans de méthodologie [23]. **Recorded Future** propose une « Geopolitical Intelligence » avec scoring de risque pays propriétaire et veille temps réel. **Bloomberg**, avec **Seerist**, a lancé des scores de risque géopolitique au niveau des entreprises. Ces offres sont fermées, orientées veille/scoring et conseil — non des **simulateurs ouverts, agentiques et reproductibles**. L'espace laissé à AI for Geopolitics est donc clair : un système **open, explicable, jouable et évaluable**, pensé pour la recherche et la démonstration d'ingénierie, là où l'offre commerciale vise la veille opérationnelle propriétaire.

10. Positionnement de AI for Geopolitics

10.1 Carte de positionnement

Deux axes structurent le paysage : **Simulation** ← **Analyse** et **Prompt-only** ← **Data-grounded**. Le projet vise le quadrant **Simulation + Data-grounded**, peu peuplé.

Prompt-only	Data-grounded (contraint par données)
-------------	--

Simulation	WarAgent, Geopol- Modeller, agents de rôle « joue la Chine »	AI for Geopolitics (cible) : agents-pays contraints + RAG + risque
Analyse	Q/R LLM sur l'actualité	GPR Index, RANE / Recorded Future / Bloomberg-Seerist

10.2 Ce qui différencie le projet

Le différenciateur n'est pas une brique unique mais leur **intégration** :

- **Agents contraints par des données structurées** (P2) plutôt que par un seul system prompt — là où la plupart des prototypes restent prompt-only.
- **RAG sourcé et citable** (P4) intégré au raisonnement des agents — absent de WarAgent et des outils ouverts.
- **Game Master + négociation textuelle visible** : dimension serious game lisible et démontrable.
- **Moteur de risque explicable** (P3) inspiré du GPR : des signaux et leurs facteurs, pas un oracle.
- **Déploiement reproductible et local-first** (P6, P9) : Docker → Compose → Kubernetes, orchestration MCP, inference locale maîtrisée.
- **Évaluation de premier ordre** (P7) : retrieval (recall@k/MRR) et comportement (alignement, sévérité, cadrage).

10.3 Lacunes de l'état de l'art que le projet adresse

- **Le déficit de validation** de la simulation sociale générative [24] : le projet en fait une exigence native (P7).
- **L'absence de couplage données structurées + RAG** dans les simulateurs LLM existants (P2, P4).
- **Le manque d'explicabilité du risque** : un risk engine transparent répond à P3 et aux préoccupations de sécurité [2][6].

10.4 Risques et garde-fous

Quatre risques et leurs parades (P8) : **biais d'escalade** des LLM [2] → suivi de la sévérité des actions, scénarios de contrôle, jamais de boucle de décision létale autonome ; **biais géopolitiques et linguistiques** [5][6] → tests multi-modèles et multi-langues, transparence des sources ; **dérive comportementale dans le temps** [5] → métriques de stabilité par round ; **usage dual** → cadrage explicite « recherche / analyse de signaux », documentation des limites (limitations.md), refus de tout cas d'usage opérationnel létal.

10.5 Le projet comme feuille de route d'AI Engineer

Au-delà de l'objet géopolitique, le projet est un **vecteur d'apprentissage** : il exerce, de bout en bout, les compétences de ta feuille de route (Tomes 1 à 5). C'est un atout portfolio

de premier plan (ESA, Applied AI, RAG / agentic engineering, Cloud / Platform), parce qu'il transforme des connaissances en un système complet, déployé et mesuré.

Compétence (Tome)	Exercée dans le projet par	Brique
T1 — Fonctionnement du modèle	Tokenization, fenêtre de contexte, gestion du cache KV, choix du modèle local	model-service
T2 — Embeddings, Vector DB, RAG [28]	Pipeline hybride Chroma + BM25 + RRF + rerank, RAPTOR / GraphRAG, métriques retrieval	rag-service
T3 — Agents, MCP, sécurité, observabilité [29]	CountryAgents + LangGraph + outils MCP + défense prompt-injection + tracing Langfuse	agents / mcp / obs
T4 — Ingénierie de prompt [30]	Prompts structurés des pays, chaîne de pensée / auto-cohérence, sortie JSON validée, calibration du risque	agent prompting
T5 — Paysage des modèles [31]	Choix du modèle, quantization, arbitrage API vs auto-hébergement, déploiement local	model strategy

10.6 Objectifs d'apprentissage prioritaires et principes d'ingénierie

Au-delà des compétences de la section 10.5, quatre **objectifs d'étude** structurent le projet :

- **Service d'inférence LLM optimisé en Python** (section 8.5) : saisir la réalité physique de l'IA générative et maximiser le rendement d'un GPU contraint.
- **Maîtrise de la conteneurisation** — l'atout majeur à démontrer. Documenter explicitement comment **Docker** (images par service, multi-stage, limites de ressources) et **Kubernetes** (kind en local : déploiements, services, ConfigMaps, secrets, probes) structurent une **architecture robuste**, dans docs/architecture.md et docs/deployment.md.
- **Architecture logicielle autour des modèles** : la valeur n'est pas de *lister* des modèles (BERT, Word2Vec) mais de concevoir l'**architecture logicielle** qui les entoure — pipelines, services, et surtout un **framework multi-agents** propre (interfaces d'agents, espace d'action, orchestration LangGraph).
- **Frameworks multi-agents** : faire des CountryAgent des objets clairs et composables, pas des prompts jetables.

Et cinq **principes d'ingénierie** non négociables, dans l'esprit « solutions simples, pas d'usine à gaz » :

- **Code propre et moderne**, aéré et lisible.
- **Orienté objet** : modèles de domaine (CountryState, WorldState, GeoEvent...) et interfaces (InferenceBackend, Agent, Retriever) bien découplés.
- **Tests** : unitaires (moteur de conséquences, parsing JSON), d'intégration (un round complet), d'évaluation (retrieval, comportement).
- **Git rigoureux** : commits atomiques, branches par feature, messages clairs, CI.
- **Simplicité d'abord** : préférer la solution la plus simple qui marche ; n'ajouter une brique (Kafka, Elasticsearch, Kubernetes) que lorsqu'un besoin réel la justifie (cf. garde-fou de méthode, section 11).

11. Des problématiques aux briques à implémenter

Ce tableau relie chaque problématique à l'état de l'art mobilisé et à la brique à construire (avec sa techno candidate), prête à être implémentée — y compris assistée par Claude Code. Il s'aligne sur une roadmap progressive : déterministe d'abord, agentique ensuite, distribuée en dernier.

Problématique	État de l'art mobilisé	Brique à construire (techno)	Phase
P1 Crédibilité	CICERO [3], WarAgent [1]	CountryAgent hybride : profil + état + mémoire + action space, sortie JSON validée (Pydantic)	1
P2 Données	World Bank, SIPRI, OEC, GDELT [11] [14][16][17]	Pipeline d'ingestion → CountryState (PostgreSQL)	0, 4
P3 Risque	GPR Index [10], Escalation [2]	Risk engine explicable + moteur de conséquences	1, 3
P4 Traçabilité	RAG-Fusion [18], reranking, GraphRAG [19]	RAG hybride (Chroma + BM25 + RRF + cross-encoder) + citations	3
P5 Mémoire	Generative Agents [4]	Mémoire 4 niveaux : Postgres + event store + vector store	1, 2
P6 Orchestration	LangGraph [21], MCP [20], Redis	Moteur de rounds (graphe d'états) + outils MCP + broker Redis	1, 6, 7
P7 Évaluation	RAGAS / TREC RAG,	Suite d'éval retrieval	3, 5

	revues GABM [7] [24]	(recall@k/MRR) + comportement (alignement, sévérité)	
P8 Éthique	Rivera [2], Red Lines [6], Strategic Actors [5]	Garde-fous, tests multi-modèles, limitations.md	Toutes
P9 Exéc. locale	Tome 5 [31], llama- cpp-python / ExLlamaV2 / vLLM [25][26][27][32] [33]	Service d'inférence Python (FastAPI + backend optimisé) + API fallback ; budget contexte ; observabilité GPU	1, 6

Garde-fou de méthode : le principal danger est l'explosion de complexité (agents + datasets + fine-tuning + MCP + Kubernetes + carte + RAG simultanément). Trajectoire saine : **Phase 0** moteur déterministe (Mesa, sans LLM) ; **Phase 1** agents LLM avec sortie structurée ; **Phase 2** diplomatie ; **Phase 3** RAG ; **Phase 4** données réelles ; **Phase 5** interface ; **Phase 6** infra (Docker → K8s) ; **Phase 7** MCP et architecture distribuée. Un MVP fidèle se limite à une zone (p. ex. mer Rouge), 6–8 acteurs, un scénario, un risk score, un dashboard simple.

12. Conclusion et prochaines étapes

L'état de l'art confirme que AI for Geopolitics est **ambitieux mais faisable**, et surtout **bien positionné** : la recherche a validé les briques (pays-agents, négociation, mémoire), l'écosystème a mûri (LangGraph, MCP, données ouvertes), l'offre commerciale laisse ouvert l'espace d'un simulateur agentique **ouvert, sourcé, explicable et évalué**, et la contrainte matérielle (RTX 2060 Super, 32 Go) est **gérable** via une pile local-first hybride. Les différenciateurs — couplage données + RAG, risque explicable, évaluation de premier ordre — répondent aux lacunes documentées du champ. En prime, le projet est un **capstone d'AI Engineer** qui exerce l'intégralité de ta feuille de route.

Trois prochaines étapes concrètes : (1) **figer le périmètre du MVP** (zone, acteurs, scénario) et les schémas de données (CountryState, WorldState, GeoEvent, AgentDecision, RoundSummary) ; (2) **implémenter la Phase 0** déterministe pour valider la boucle de rounds et le moteur de conséquences avant tout LLM ; (3) **rédiger les documents cadres** (architecture.md, simulation_model.md, data_governance.md, limitations.md). Ces étapes se prêtent à une implémentation assistée par Claude Code, brique par brique, en suivant le tableau de la section 11.

Références

- [1] Hua W., Fan L., et al. — War and Peace (WarAgent): LLM-based Multi-Agent Simulation of World Wars (2023). arXiv:2311.17227. <https://arxiv.org/abs/2311.17227>
- [2] Rivera J.-P., Mukobi G., Reuel A., Lamparth M., Smith C., Schneider J. — Escalation Risks from Language Models in Military and Diplomatic Decision-Making. FAccT 2024. arXiv:2401.03408. <https://arxiv.org/abs/2401.03408>
- [3] Meta FAIR (Bakhtin et al.) — Human-level play in the game of Diplomacy (CICERO). Science, 2022. <https://ai.meta.com/research/cicero/>
- [4] Park J.S., O'Brien J., Cai C.J., Morris M.R., Liang P., Bernstein M.S. — Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. arXiv:2304.03442. <https://arxiv.org/abs/2304.03442>
- [5] LLMs as Strategic Actors: Behavioral Alignment, Risk Calibration, and Argumentation Framing in Geopolitical Simulations (2026). arXiv:2603.02128. <https://arxiv.org/abs/2603.02128>
- [6] Red Lines and Grey Zones in the Fog of War: Benchmarking Legal Risk, Moral Harm, and Regional Bias in LLM Military Decision-Making (2025). arXiv:2510.03514. <https://arxiv.org/abs/2510.03514>
- [7] Do Large Language Models Solve the Problems of Agent-Based Modeling? A Critical Review of Generative Social Simulations (2025). arXiv:2504.03274. <https://arxiv.org/abs/2504.03274>
- [8] Agentic Large Language Models, a survey (2025). arXiv:2503.23037. <https://arxiv.org/abs/2503.23037>
- [9] Rosehill D. — Geopol-Modeller : simulation multi-acteurs LLM pour la prospective géopolitique (basée sur Snowglobe). GitHub. <https://github.com/danielrosehill/Geopol-Modeller>
- [10] Caldara D., Iacoviello M. — Measuring Geopolitical Risk. American Economic Review, 112(4), 2022. (GPR Index) <https://www.matteoiacoviello.com/gpr.htm>
- [11] The GDELT Project — Global Database of Events, Language and Tone & Global Knowledge Graph. <https://www.gdeltproject.org/>
- [12] ACLED — Armed Conflict Location & Event Data Project. <https://acleddata.com/>
- [13] UCDP — Uppsala Conflict Data Program. <https://ucdp.uu.se/>
- [14] SIPRI — Stockholm International Peace Research Institute (bases militaires). <https://www.sipri.org/databases>
- [15] Correlates of War Project. <https://correlatesofwar.org/>
- [16] World Bank Open Data. <https://data.worldbank.org/>

- [17] The Observatory of Economic Complexity (OEC). <https://oec.world/>
- [18] Rackauckas Z. — RAG-Fusion: a New Take on Retrieval-Augmented Generation (2024). arXiv:2402.03367. <https://arxiv.org/abs/2402.03367>
- [19] Microsoft Research — Project GraphRAG.
<https://www.microsoft.com/en-us/research/project/graphrag/>
- [20] Anthropic — Introducing the Model Context Protocol (2024).
<https://www.anthropic.com/news/model-context-protocol>
- [21] DataCamp — CrewAI vs LangGraph vs AutoGen: Choosing the Right Multi-Agent AI Framework (2025). <https://www.datacamp.com/tutorial/crewai-vs-langgraph-vs-autogen>
- [22] Project Mesa — Agent-based modeling in Python (et Mesa-Geo).
<https://github.com/projectmesa/mesa>
- [23] RANE / Stratfor ; Recorded Future Geopolitical Intelligence ; Bloomberg + Seerist.
<https://www.ranenetwork.com/>
- [24] Validation is the central challenge for generative social simulation: a critical review of LLMs in agent-based modeling (2025). PMC.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12627210/>
- [25] Ollama — exécution locale de LLM (Windows / Linux / macOS). <https://ollama.com/>
- [26] llama.cpp — inference de LLM quantifiés (GGUF).
<https://github.com/ggerganov/llama.cpp>
- [27] vLLM — moteur d'inference à haut débit. <https://github.com/vllm-project/vllm>
- [28] Feuille de route de l'AI Engineer — Tome 2 : Recherche sémantique, bases vectorielles et RAG (2026).
- [29] Feuille de route de l'AI Engineer — Tome 3 : Agents, outils et MCP, sécurité et observabilité (2026).
- [30] Feuille de route de l'AI Engineer — Tome 4 : L'ingénierie de prompt (2026).
- [31] Feuille de route de l'AI Engineer — Tome 5 : Le paysage des modèles (2026).
- [32] llama-cpp-python — bindings Python de llama.cpp (serveur d'inférence GGUF).
<https://github.com/abetlen/llama-cpp-python>
- [33] ExLlamaV2 — inférence rapide de LLM quantifiés (EXL2/GPTQ) sur GPU grand public.
<https://github.com/turboderp-org/exllamav2>
- [34] Hugging Face Transformers + bitsandbytes — chargement et quantization 4-bit/8-bit.
<https://huggingface.co/docs/transformers/quantization/bitsandbytes>
- [35] FastAPI — framework d'API Python pour exposer le service d'inférence.
<https://fastapi.tiangolo.com/>